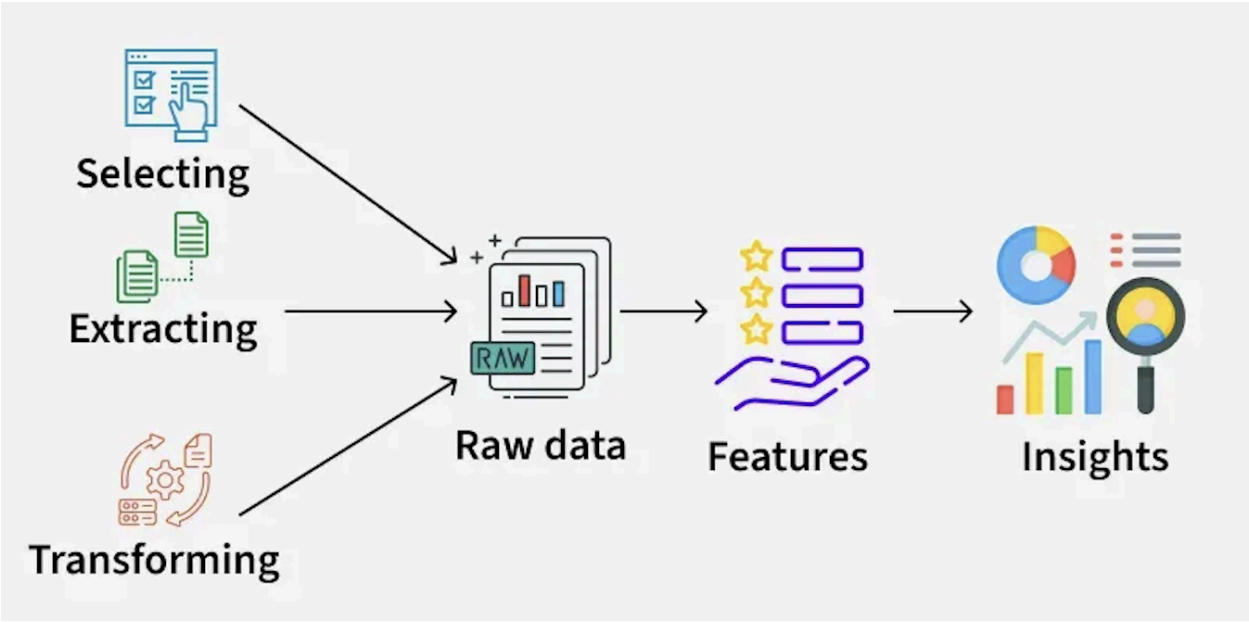
To see full blog click the Link

What if your model is already perfect and the only thing holding it back is your data?



Feature engineering is the process of transforming raw data to relevant information for machine learning models to use. In simple words, feature engineering is creating predictive features for the models. The model's performance largely depends on the input variable feed into the machine during training, feature engineering is a crucial preprocessing step which needs selective and most relevant data for predicting values.

This concept plays an important role in machine learning because machine learning models cannot work with raw data, so we need to manually transform them into numeric form so the model can understand, which means converting images and text into numeric form. Feature engineering is important because it improves efficiency of models and focuses more on relevant information that will evaluate performance.

In machine learning there is a common principle known as **garbage in garbage out (GIGO),** which refers to the idea that the quality of output given by a system is determined by the quality of input. Imagine you're building a model to predict house prices. Helpful features that generate meaningful results would include **house size, number of rooms, and location.** These features have relation with price so the model will predict more accurate results however, garbage

features like ID numbers, landlord name have no relation with prices so including them in training will confuse the machine and produce useless information.

Common Feature Engineering Technique

1. **Feature Encoding**

   As the principle suggests, better encoding leads to better prediction. Algorithms cannot read categorical data unless they are transformed into numeric values.

   Dataset: To download the dataset click on this Link

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
df = pd.read_csv('featureeng.csv')
df.head()

df_encoded = df.copy()
label_encoder = LabelEncoder()
```

Label Encoding

Label encoding converts category values to number, either Yes to one and No to zero

```
df_encoded["Promotion Enc"] = label_encoder.fit_transform(df_encoded["Promotion"])
df_encoded.head()
```

One Hot Encoding

One hot encoding creates a binary column for each category, unlike label encoding this will also avoid giving the wrong label to the model.

```
df_encoded = pd.get_dummies(df_encoded, columns = ["City"], prefix = ["City"])
df_encoded.head()
```

Frequency Encoding

Frequency encoding will replace the category with the number of times a value appears in the dataset.

```
job_title_counts = df_encoded["JobTitle"].value_counts().to_dict()
df_encoded["JobTitle_Enc"] = df_encoded["JobTitle"].map(job_title_counts)
df_encoded.head()
```

Target Encoding

Target encoding replaces the value with the mean variable of the targeted category.

```
education_target = df_encoded.groupby("Education")["Promotion Enc"].mean().to_dict()
df_encoded["Education_Target_Enc"] = df_encoded["Education"].map(education_target)
df_encoded.head()
```

2. **Feature Scaling**

We import libraries StandardScaler, MinMaxScaler from scikit-learn, to standardize features using mean, scaling unit variance and scale feature to fix range.

Dataset: To download the dataset click on this Link

```
import pandas as pd

from sklearn.preprocessing import StandardScaler, MinMaxScaler
df = pd.read_csv('featureeng.csv')
standardscaler = StandardScaler()
minmax = MinMaxScaler()

df['salarystdscaler'] = standardscaler.fit_transform(df[['Salary']])
df['salaryminmax'] = minmax.fit_transform(df[['Salary']])

df[['Salary','salarystdscaler','salaryminmax']].describe()
```

3. **Feature extraction**

Feature extraction means to use already existing data to create new features which will help the model learn better and predict better results.

```
import pandas as pd
df  = pd.read_csv('customer.csv')
df.head(3)
```

a new column is cut from column age where bins is seperated values to group and label is labelling them 10, 30, 50, 70 are the values where 10 is young, 30 and 50 is mid and 70 is senior

```
bins = [10,30,50,70]
labels = ["young","mid","senior"]
df["age_group"] = pd.cut(df["age"], bins = bins, labels = labels)

df[["age","age_group"]].head()
```

extract year and month from dataset to create new columns and use today's date and sign in date to find the total date for sign in same with last purchase this shows how long the customer hasn't purchased anything

```
df['signup_year'] = df['signup_date'].dt.year
df['signup_month'] = df['signup_date'].dt.month
```

```
df['date_since_signin'] = (pd.Timestamp.today() - df['signup_date']).dt.days
df['date_since_lastpur'] = (pd.Timestamp.today() - df['last_purchase_date']).dt.days
print(df['date_since_signin'].head(3))
print(df['date_since_lastpur'].head(3))
```

apply lambda function from start to end split the values and 0 means select the first word and label as name and -1 is label for last values which is now name lastname and length of name

```
df['first_name'] = df['name'].apply(lambda x:x.split()[0])
df['last_name'] = df['name'].apply(lambda x:x.split()[-1])
df['length_name'] = df['name'].apply(len)

print(df['first_name'].head(3))
print(df['last_name'].head(3))
print(df['length_name'].head(3))
```

if the total spend of custon=mer is more then 5000 then theu are high spender which is 1 and if its low it will be 0|

```
df['high_spender'] = (df['total_spent'] > 5000).astype(int)
print(df['high_spender'])
df.head(3)
```

Want to see the full code? All the code used in this post is available on my GitHub. Feel free to explore, clone, and experiment with it yourself.

[View on GitHub]